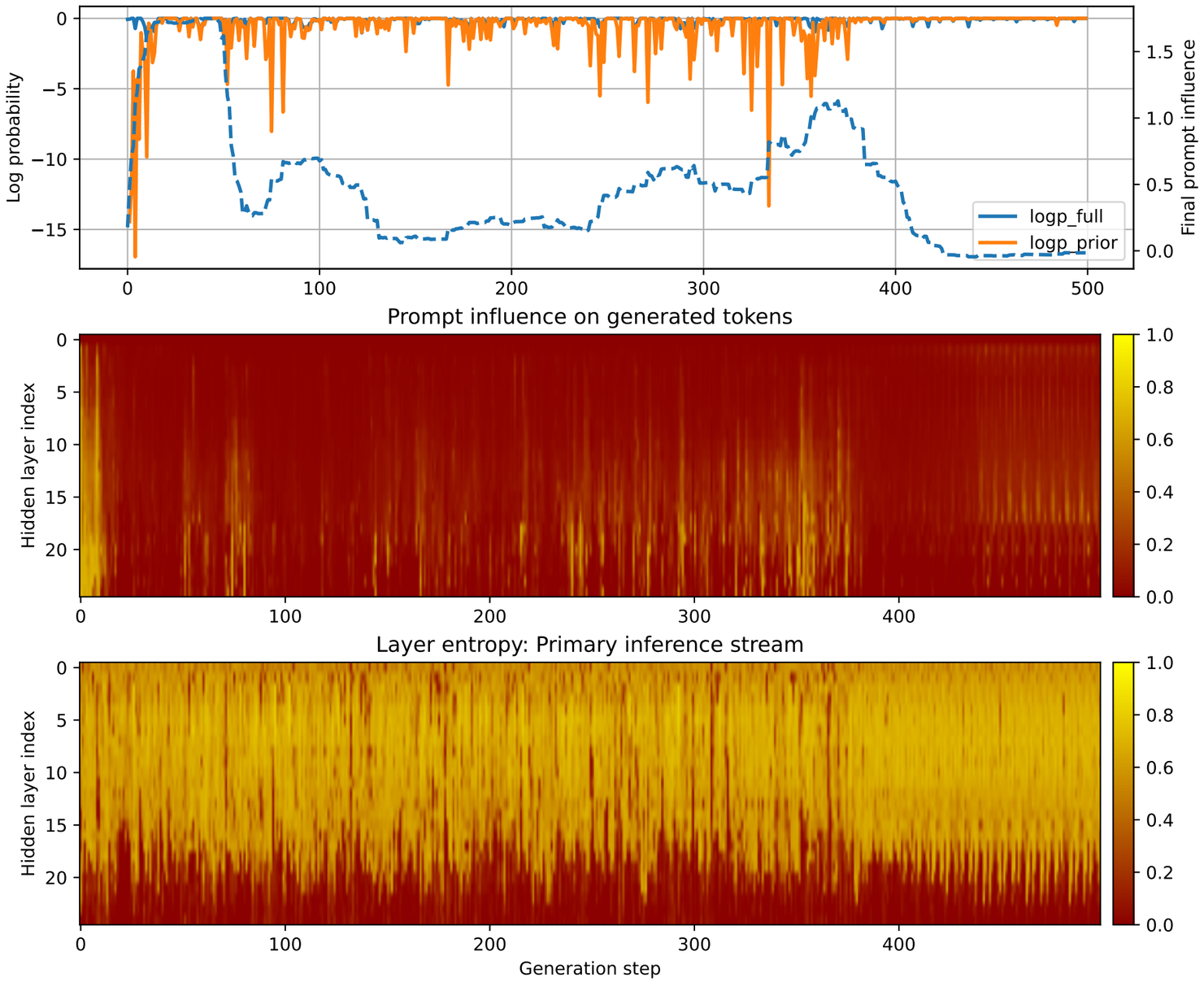
Prompt 1




```
PROMPT:
Task ID: PYC064  Return ONLY valid Python code. Do not use
Markdown fences. Do not include prose,
explanations, examples, print statements, or tests.  Your code
must contain this exact check-token line: #
CHECK_TOKEN: PYC064  Your code must contain this required symbol:
def shortest_path_unweighted  Implement this
task: define function
shortest_path_unweighted(graph,start,goal)..  Start your answer
with the check-token
line, then provide the Python implementation.

MODEL RESPONSE:
# CHECK_TOKEN: shortest_path_unweighted  # Definition for a
graph. # class Graph: #     def __init__(self,
vertices): #         self.graph = {} #         for i in
range(vertices): #             self.graph[i] = []  #
Function to calculate the shortest path between two nodes in an
unweighted graph using Dijkstra's algorithm.
def shortest_path_unweighted(graph, start, goal):     # Create a
dictionary to store the shortest path
distances from the start node to each node in the graph
distances = {node: float('inf') for node in graph}
# Initialize the distance of the start node to 0 and the distance
of all other nodes to infinity
distances[start] = 0     for node in graph:
distances[node] = float('inf')      # Create a priority
queue to store the nodes to be visited and their corresponding
distances     queue = [(start, 0)]      # While
the priority queue is not empty:     while queue:         # Get
the node with the smallest distance from the
start node         current_node, current_distance =
heapq.heappop(queue)          # If the current node is the
goal node, return the shortest path distance         if
current_node == goal:             return
current_distance          # For each neighbor of the current
node:         for neighbor, distance in
graph[current_node].items():             # If the neighbor is not
in the priority queue or its distance is
less than the current distance, update the distance and add it to
the priority queue             if neighbor
not in queue or distance < distances[current_node] + distance:
            distances[neighbor] = distance
heapq.heappush(queue, (neighbor, distance))      # If the goal
node was not found, return -1     return -1  #
Example usage: graph = {     'A': ['B', 'C'],     'B': ['D',
'E'],     'C': ['F'],     'D': ['G'],     'E':
['H'],     'F': ['I'],     'G': ['J'],     'H': ['K'],     'I':
['L'],     'J': ['M'],     'K': ['N'],
'L': ['O'],     'M': ['P'],     'N': ['Q'],     'O': ['R'],
'P': ['S'],     'Q':
```